Safe Sweden Platform

Simplified Technical Reflection & Implementation Direction

This document condenses Claude's codebase analysis into a practical roadmap. The goal is to keep the project understandable and implementable while preserving the strongest AI Agent and Platform Engineering opportunities.

1. What the project has today

• **Frontend:** React + Vite + Tailwind, with authentication, crime browsing, saved crimes, charts, and a single-crime map.

• **Backend:** FastAPI + SQLAlchemy + MySQL, JWT authentication, crime/profile/auth routes.

• **Deployment:** Frontend on Vercel, backend/database on Railway.

• **Main gap:** several visible features are still stubs or hardcoded, especially Statistics, Zones, and Crime History.

• **Platform gap:** no proper CI/CD, tests, Docker, observability, background jobs, or infrastructure-as-code.

2. First: fix the foundation

Before adding sophisticated features, address the issues that could make the platform unsafe or difficult to maintain.

Priority	Action	Why
Urgent	Rotate production database credentials and secure SECRET_KEY	Credentials were found in the repository; this is a security issue.
High	Remove DATABASE_URL from logs	The connection string may expose credentials.
High	Add ownership checks to crime update/delete	Authenticated users should not be able to modify other users' records.
High	Add backend tests	There are currently no meaningful test files.
High	Add CI/CD	Automatically test before deployment.
Medium	Dockerize the project	Makes local setup and deployment more reproducible.

3. Keep the new features focused

Feature	Simple goal	Portfolio value
Map Coverage	Turn Zones into a real map and progressively add Swedish municipalities/regions.	Strong visual + data engineering
Community Safety	Replace Crime History with a useful timeline and introduce a simple area safety score.	Strong product value
AI Informing	Start with AI-generated trend explanations, then build one real tool-using Safety Intelligence Agent.	Very high AI Agent value
Live Alerts	Start with email alerts/digests; later add real-time geographic alerts.	Strong event-driven/platform value

Feature	Simple goal	Portfolio value
Platform Foundation	Tests + CI/CD + Docker + logging/metrics + Redis when needed.	Very high Platform Engineering value

4. The AI strategy: one strong agent, not many agents

The most valuable AI direction is a **Safety Intelligence Agent**. Instead of making a generic chatbot, let the agent use the platform's own APIs as tools.

• User asks a complex safety question.

• Agent decides which platform tools it needs.

• Tools query crime/trend/area data.

• Agent combines the results and explains them in plain language.

• The answer is grounded in returned data rather than invented facts.

• The agent remains read-only at first; anything that could publish an alert requires human approval.

Important: deterministic code should handle calculations, filtering, deduplication, and validation. AI should handle reasoning, synthesis, explanation, and ambiguous cases.

5. Recommended implementation order

Step	Build	Outcome
1	Security fixes	Safe foundation
2	Tests + CI/CD + Docker	Reliable development/deployment workflow
3	Real Statistics API + Safety Score	Replace hardcoded statistics
4	Crime History	Replace the existing stub
5	Zones / Map Coverage	Real interactive municipality map
6	AI Trend Summarisation	First practical AI capability
7	Safety Intelligence Agent	Main AI Agent showcase
8	Logging + metrics + Redis	Production/platform maturity
9	BRÅ data pipeline	Real broader Swedish coverage
10	Email/real-time alerts	Event-driven safety notifications

6. Architecture direction

Do **not** turn the project into microservices just to demonstrate architecture skills. The analysis recommends keeping FastAPI as a modular monolith. Add Redis and background workers only when a real requirement appears.

• React SPA → FastAPI → MySQL remains the core.

• Redis is introduced for caching, rate limiting, and real-time event distribution when needed.

• A scheduled/background worker handles data ingestion, digests, and anomaly processing.

• AI agents live inside a clear agent/tool layer and call controlled backend tools.

• If the platform eventually becomes genuinely national-scale, ingestion and alert dispatch can be separated into dedicated processes.

7. What to avoid for now

• Do not build RAG, vector databases, or multiple specialized agents immediately.

• Do not implement web push before simpler alert delivery is working.

• Do not introduce microservices without a concrete scaling problem.

• Do not build every feature in the original proposal simultaneously.

• Do not make AI responsible for deterministic calculations or safety-critical publishing decisions.

8. The core portfolio story

A strong final project story would be: **a real civic-tech platform that evolved from a full-stack application into a production-aware platform with reliable data, observable infrastructure, and a genuine tool-using AI agent.**

That is more compelling—and much easier to implement and explain—than trying to demonstrate every possible modern technology.